Detecting Sudden Drift for Single-instance Log in Software Development Process

Jiayi Yuan[1,a], Tong Li[2,3], Rui Zhu[1,2,*], Zifei Ma[1], Jiayuan Peng[1], Yue Huang[1]
[1]School of Software, Yunnan University, Kunming, China
[2]Yunnan Key Laboratory of Software Engineering, Yunnan University, Kunming, China
[3]School of Big Data, Yunnan Agricultural University, Kunming, China
[a.]j.y.yuan@outlook.com, [*] Corresponding author, rzhu@ynu.edu.cn

***Abstract*—The current algorithm for detecting drift in the business process can only detect the drift of the log of multi-instance. However, the log generated in the software development process is mainly single-instance. Hence, this paper proposes an algorithm for detecting sudden drift of the log of single-instance. First, we apply the LDA topic model for the topic extraction of log in the software development process to obtain the log of single-instance. Then the location of the drift point of the single-instance log can be detected according to the change of the lowest frequency of direct successor relation. Finally, the location numbers of the actual drift points were obtained by corresponding to original data. Multiple sets of simulated data evaluate this algorithm. The results show that this algorithm can effectively detect the sudden drift location of the single-instance log in the software development process.**

Keywords—software development process, process drift, LDA topic model, single-instance, frequency detection

I. INTRODUCTION

The core issue of process mining is to mine the model from the event log to monitor and improve the information systems' execution. Process mining is divided into three scenarios: process discovery, conformance checking, and process enhancement [1]. Specifically, process discovery is to mine the process model complying with the real execution of the system from the event log. The traditional algorithms of process discovery assume that the process model is to be static, whereas, in the real execution of the system, the process model is always dynamic [2], which may be occurred process drift. It is essential to solving the model evolution caused by the process drift to mine the process model more complying with the real process. The process drift detection algorithm is to solve this problem. Determining how many times evolutions of process model occur in real execution and how it happens respectively by a series of detection approaches like feature extraction, thereby to serve the process discovery better. The process drift consists of four categories: sudden drift, gradual drift, recurring drift, and incremental drift [2]. Among them, sudden drift refers to the evolution before and after a certain time point caused by the process model is inconsistent. Gradual drift refers to the evolution before and after a period of time, and the model evolves to another model gradually during this period of time; several models maybe also occur at the same time [3]. This paper mainly focuses on the algorithm for detecting sudden drift in the software development process.

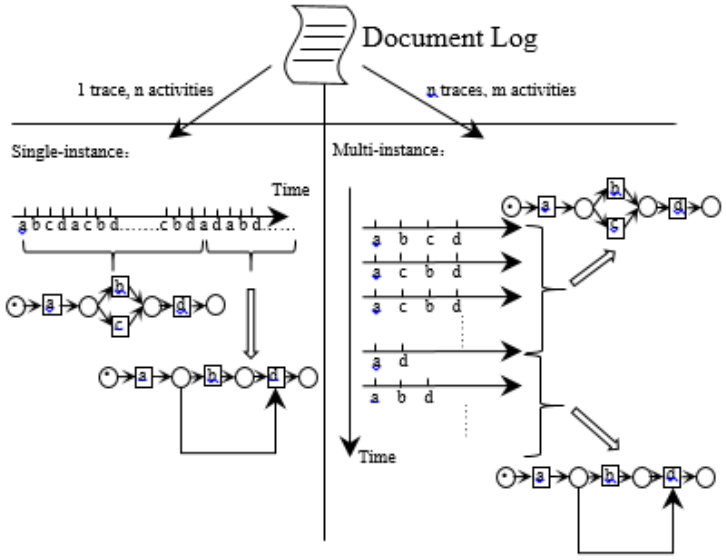

Fig. 1. The difference between Single-instance and Multi-instance.

2166-5656/20/$31.00 ©2020 IEEE
DOI 10.1109/PHM-Besancon49106.2020.00069

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:40:55 UTC from IEEE Xplore. Restrictions apply.

The current process drift detection algorithms mainly can detect multi-instance log [4-9], whereas, the log recorded in the software development process is mostly single-instance log due to the lack of case number [10], which cannot be output as the drift point with case number when being detected. In other words, there is only one case in the single-instance log, and the minimum unit of drift detection must be the activity level instead of the case level. The other difference between single-instance and multi-instance log is the different growth patterns of data, which is the growth in the length of one trace and the number of multiple traces in unchangeable lengths, respectively. The comparison is shown in Figure 1. In the left part of Figure 1, there is one trace with the unlimited growth of length and the unknown quantity of activity. When concept drift occurs, the change point shall be detected in activities. In the right part, the amount of trace is in an unlimited increase, and each of them is the combination of an unknown amount of the same set of activity. When concept drift occurs, the change point shall be detected in cases. Thus, the drift detection of the single-instance log requires the output of the change point is the trace length or the activity location.

Based on the above problem, an algorithm for detecting sudden drift in the software development process is proposed in this paper, which consists of two stages. First, the LDA topic model [11] is used to extract the single-instance log to be detected; Second, the change of the lowest frequency of the direct successor relation of activities in single-instance data detects the drift location. Besides, the effectiveness of this algorithm is verified by multiple sets of simulated data.

Other sections of this paper are arranged as follows: Section 2 is the current work related to process sudden drift; Section 3 describes the basic definitions involved; the approach is detailed in Section 4; the algorithm is verified in Section 5; finally, Section 6 is the conclusion and further study.

II. Related work

Concept drift first appeared in machine learning [12] to solve the problem that the classifiers cannot classify well with the passage of time and increasing data size. It is a relatively new research direction in process mining. The current work related to the detection algorithm of sudden process drift is reviewed as follows.

Bose et al. [2] first proposed the existence of concept drift in process mining. They have analyzed the real process of more than 100 organizations, and the results show that it is very unrealistic to assume that the process being studied is in a steady state. This change is seldom recorded directly in the log, which requires to be detected by the algorithm. Later, they proposed a framework [4] that extracting feature and hypothesis testing to detect drift, which is the mainstream drift detection algorithm. However, feature extraction is hugely complicated. Subsequently, Martjushev et al. [5], based on this, proposed a self-adaptive sliding window to solve the problem that only one step of the window can be moved when extracting features. And then, Maaradji et al. [6] put forward an approach, which is by extracting the features of the run and chi-square testing. The characteristics of the run are to compress log by concurrent relation, thus reducing the complexity of extracting features. But the shortcoming is that concurrent relation is hard to determine rapidly. Kumar et al. [7] used event classes and hypothesis testing to detect the concept drift. However, it works not well in some of the change patterns and settings of log size.

Recently, Zheng et al. [8] adopted the activity relation matrix and DBSCAN clustering algorithm; Lin et al. [9] used the local completeness of direct successor relation and the skipping forgetting mechanism of windows by judging disappear and increase of relations to detect the drift. Their approaches can detect the drift rapidly and accurately, and only use the direct successor relation of activities significantly reduced the complexity of feature extraction. As well as, their approaches also avoid the problem that the sliding window can be moved just one step ever time. However, the above algorithms are mostly suitable for the multi-instance event log. For the single-instance event log, it must be appropriately divided into the multi-instance event log to be detected. But the appropriate partition of the log is also a challenge. This paper proposes an algorithm for detecting sudden drift of single-instance event log in the software development process without partition of the log. It can directly detect the drift without appropriately partition the single-instance log.

III. Preliminaries

Preliminaries involved in this paper consist of basic definitions and the principle of the LDA topic model [11].

A. Basic concept definitions

The process model can be expressed in multiple forms, and the expression form of Petri-net [13] is adopted in this paper. The basic definition of Petri-net is presented as follows.

Definition 1 (Petri nets) [9]. A Petri-net is a 4-tuple $N=(P, T, F, M_0)$, where,

(1) $P\cup T\neq\varnothing$ and $P\cap T=\varnothing$, P is a finite set of places, and T is a finite set of transitions;

(2) $F\subseteq(P\times T)\cup(T\times P)$, F represents the relation of flow;

(3) $dom(F)\cup cod(F)=P\cup T$, and $dom(F)=\{x\in P\cup T|\exists y\in P\cup T:(x, y)\in F\}$, $cod(F)=\{y\in P\cup T|\exists x\in P\cup T:(x, y)\in F\}$;

(4) $M: P\rightarrow\{0,1,2,\ldots\}$, M is called the marking of P, and M_0 is called the initial marking of P.

The approach is proposed for the single-instance event log, so the relevant definitions of event log are given.

Definition 2 (event log and trace) [9]. Let T be the set of all tasks; $\sigma\in T^*$ is an event trace, and $W\subseteq\mathcal{P}(T^*)$ is an event log.

From Definition 2, it can be known that event log is the set of traces, and each of them is arranged by the sequence of limited tasks (also called activities).

Definition 3 (direct successor relation, >) [9]. Let T be a set of tasks, and $W\subseteq\mathcal{P}(T^*)$ is the event log. For $\forall a, b\in T$, $a>b$, if and only if a trace $\sigma=t_1\ t_2\ t_3\ldots t_{n-1}\ t_n$ exists, and $i\in\{1,\ldots, n-1\}$, it satisfies $\sigma\in W$ and $t_i=a$, $t_{i+1}=b$.

It can be known from Definition 3 that the direct successor relation is adjacent relation.

Definition 4 (single-firing sequence) [14]. Let W be an

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:40:55 UTC from IEEE Xplore. Restrictions apply.

event log; a trace $\sigma \in W$ is called a single-firing sequence, if and only if $W = [\sigma^1]$.

It can be known from Definition 4 that a single-firing sequence (also called single-instance log) is the event log consisting of only one trace.

B. The principle of LDA topic model

LDA topic model [11], the full name is Latent Dirichlet Allocation, is the three-level hierarchical Bayesian model, "document-topic-word", to infer topics from the documents collected [15], which is shown in Figure 2. It can present the topics in the document in the form of the probability distribution. A document can involve multiple topics and each word of the document is from one of the topics.

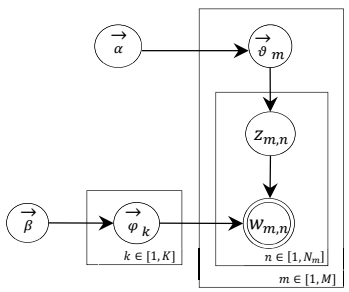

Fig. 2. The principle diagram of LDA [11]

In Figure 2, w only is the variable observed, and others are latent variables or parameters. Among them, φ represents word distribution, ϑ is topic distribution, α is the parameter of the prior distribution (i.e., Dirichlet distribution) of topic distribution ϑ, and β is the parameter of the prior distribution (i.e., Dirichlet distribution) of word distribution φ. And, w represents words, all words; z represents topics, preset. N is the total number of words in a document, and M is the total number of documents in the corpus, and K represents the number of topics.

For each word in a document, LDA determines the topic distribution ϑ_m of a document such as m according to prior knowledge α. Then, extracting a topic from topic distribution ϑ_m (multinomial distribution) corresponded with the document such as the topic $z_{m,n}$ of the n-th word in document m. Next, determining the word distribution φ_k of the current topic according to prior knowledge β. Subsequently, extracting a word $w_{m,n}$ from word distribution φ_k corresponded with topic $z_{m,n}$. After repeating the process for N times, the document generates.

IV. Drift Detection

The approach includes two stages. First, extracting the single-instance activity sequence of event log in the software development process by using LDA. Second, detecting the drift in the extracted sequence of activities by adopting the lowest frequency change of direct successor relation.

A. The extraction of activity sequence

The first stage is extracting the single-instance log. The log needs to be extracted first due to the lack of activity attributes. The traditional activity extraction approach is to map the activity of software development, such as DES, CODE, TEST, REV, and VER, according to the suffix of filenames of each event in the log and a certain of regular expressions [16]. However, there are two disadvantages to this method. One is that the rules of regular expressions need to define artificially. For example, the rule provided in [16] is only suitable for the event log in the development process of software programmed in java language. The rule needs to be rewritten according to the actual situation of log for other programming languages like python and c-language. Another is the number of activities mapped is small and fixed. Whatever software development log is, all mapped to six activities of the above software development life cycle, whereas the real development process includes more than these basic activities of six. In this paper, the extraction of activity of the software development process by LDA can solve the problem.

Assume that the content recorded by the log is divided into multiple topics (that is, multiple activities or tasks), each record is a document, and the contents of each record is multiple words serve one topic in the form of the multinomial distribution, and the multiple words under each topic are also in multinomial distribution. Thus LDA can be used to extract the topic distribution and the probability distribution of the words under each topic, that is, the "topic-word" distribution.

This paper extracted the log of the Tensorflow project on Github, and one master branch log[1] with 16687 records is obtained, with the record time is from November 6, 2015, to March 23, 2019. Table 1 shows the form of the log. One line is a record of behavior at one time, including the serial number, author, submission time, and submission instructions.

Detecting drift is oriented to the time-sequenced activity sequence. Therefore, we extract the topic of the submission instructions of the log according to the approach proposed in this paper. The results are shown in Table 2, which gives the top ten words with the highest probability corresponding to each topic when the number of selected topics is eight. Among them, the topic is the activity that we need to map, and we map the corresponding topics according to the words contained in each line of the submission instructions. For example, if the submission instruction is "compat: Update forward compatibility horizon to 2019-03-23", the topic mapped "h" from the topic extraction results in Table 2. If the word corresponding to any topic is not included in the submission instruction, no mapping is performed. This line of record can only be discarded, and the sequence of other activities are not affected. Python language provides a library of LDA functions that can be called directly. Users can also choose the number of generated topics by themselves. The number of topics is the number of activities that need to be mapped eventually. Extracting events by LDA solves the problem that the rule is defined artificially, and the number of activities is small and fixed in approaches of traditional activity extraction.

[1]https://github.com/tensorflow/tensorflow

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:40:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I. THE LOG SAMPLE OF TENSORFLOW

Number	Author	Date	Submit Instructions
36f817a9f3	A．Unique TensorFlower	Sat Mar 23 02:02:24 2019 -0700	compat: Update forward compatibility horizon to 2019-03-23
840022b021	Guangda Lai	Sat Mar 23 00:28:06 2019 -0700	Support 0-D ITensors in TF-TRT. This is required to import #25624 which adds CombinedNonBatchSuppression support.
1e4725cf3b	Randy Dodgen	Fri mar 22 21:26:46 2019 -0700	Automated rollback of commit 359e8c791c
⋮	⋮	⋮	⋮

TABLE II. THE EXAMPLE OF TOPIC EXTRACTION

Topic	The top ten words with the highest probability
a	"revert", "cleanup", "contrib", "logic", "eigen" "quantize", "modify", "annotation", "core", "algorithm"
b	"improve", "Change", "error", "type" "message" "list", "add", "do", "output", "use"
c	"number", "requirement", "class", "set", "name", "gpu", "clean", "give", "shape", "exist"
d	"add", "change", "module", "switch", "index", "test", "conversion", "xlum", "loss", "support"
e	"doc", "fix", "python", "gradient", "path", "colocate", "format", "warning", "cuda", "issue"
f	"address", "value", "include", "node", "cudnn", "array", "op", "default", "have", "control"
g	"check", "remove", "context", "apus", "API", "try-block", "TensorBoard", "base", "client", "contextlib"
h	"file", "code", "replace", "library", "Updata", "call", "pbtxt", "get", "runtime", "kernel"

B. The lowest frequency detection drift

The goal of this stage is to detect the drift directly without finding an appropriate classification rule, that is, instead of dividing the single-instance log lacking case attribute into the multi-instance log with case attributes. The approach is to detect the drift by the change of the lowest frequency of the adjacent relation of the activity sequence extracted.

The core problem of sudden drift detection is to judge the similarity of the models corresponding to the two sub logs before and after a time point. The more dissimilar, the more the drift is likely to occur. The idea given in [9] is first to determine whether the previous sub log conforms to local completeness, that is, to determine whether all adjacent relations appearing in the model have all appeared in the trace of sub log. If all appear, new relations appearing in sub logs subsequent are the log generated for the new model, instead of the same previous model, that has drifted. Inspired by this idea, this paper provided an algorithm to calculate the change of the lowest frequency in adjacent relation between the activities as activities increase in the log for the trace growing in the software development process. The location where the lowest frequency varies is the location where new relation appears, that is, the position of the drift point.

If the lowest frequency has not changed, which is 1, indicating that there have been new relationships appeared all the time, but it cannot be judged whether it is the result of the model change caused by the drift; or that the model has not changed at all, while all possible traces are not yet fully executed, that is, not all adjacent relations have appeared, that also is, no local completeness. In this way, no judgment can be made whether there is a drift or not.

Algorithm 1. ***Lowest Frequency Change Detection***

Input: Single instance activity sequence S.
Output: A set of drift points P.

1. **begin**
2. x, y, z_1, z_2, p, $q \leftarrow 0$, $R \leftarrow \emptyset$, $n \leftarrow$ length of S
3. **for** $i \leftarrow 0$ **to** n **do**
4. $R \leftarrow (S[i-1], S[i])$ // Ergodic adjacent relation
5. **if** $R[i]$ not in R **then**
6. $x \leftarrow 1$
7. **else** $x \leftarrow x + 1$
8. **for** $j \leftarrow 0$ **to** x **do**
9. **If** $x > z_1$ **then**
10. $z_1 \leftarrow x$
11. **If** $x < z_2$ **then**
12. $z_2 \leftarrow x$
13. $Z_2 \leftarrow \{z_2\}$, $m \leftarrow$ length of Z_2, $y \leftarrow$ length of $R[i]$
14. **for** $k \leftarrow 1$ **to** m-1 **do**
15. **if** $Z_2[k] = 1$ **and** $Z_2[k\text{-}1] > 1$ **then**
16. $q \leftarrow k + 1$
17. **while** $q < m$ **and** $Z_2[q] = 1$ **do**
18. $q \leftarrow q + 1$
19. **if** $q < m$ **and** $Z_2[q] > 1$ **then**
20. $p \leftarrow y$
21. $P \leftarrow \{p\}$
22. **end**

After the lowest frequency starts to increase, it suddenly drops to 1 and then slowly returns to the original pattern of growth, which is most likely due to the sudden change caused by drift. Assume that the probability of selecting the activity of each branch is random and equal when the model is executed. Then, after the lowest frequency starts to increase, the adjacent relation of the traces that may be generated in the model have all appeared, it may also before the increase. Later, if the lowest frequency is decreased to 1 and then increased, it can be considered that the model has drifted.

Let the count of adjacent relation between activities is x, and the step size of trace ever-increasing is y, as the step size increases, the highest frequency of the current adjacent relation R is z_1, and the lowest frequency is z_2. Then an algorithm can be given to detect the drift by the change of the lowest frequency of direct successor relation.

The core part of Algorithm 1 is at lines 13-21, finding the sudden change point of the lowest frequency suddenly dropping to 1 and then gradually increasing, which is the drift point, and finally, obtaining the set of all drift points. Lines 2-4 include ergodic adjacent relations, and lines 5-7 record the number of adjacent relations, and lines 8-12 are finding the lowest and highest frequency.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:40:55 UTC from IEEE Xplore. Restrictions apply.

V. Experimental Evaluation

The experimental evaluation of this paper is divided into two parts. One is to set one change pattern and a drift point to detect the effectiveness of the algorithm first; the other one is to evaluate the feasibility of the algorithm for multiple models, multiple sets of simulation data, and multiple drift points.

A. Evaluation of effectiveness

Using Algorithm 1 to test the simulated data of 17 activities with a sequence length (i.e., step size) of 20857. The drift change pattern [6] is that a conditional fragment changed to a sequential fragment (CF) and the drift position is 10317. The results show that the position of the drift point is well detected. In Figure 3, the blue line reflects the changing trend of the lowest frequency with the increase of the step size and the number of relations, and the red line is the changing trend of the highest frequency. It can be seen that the blue line has risen slowly after a sudden drop at around 10,000, while the red line has been on an upward trend since it starts to grow. The position of the drift point output by the algorithm is 10317, which is a correct result, thus verifying the effectiveness of this algorithm. It can also be seen in Figure 3 that the number of relations tends to a final value and no longer grows at a position of about 40. The actual number of relations in this simulated data is 39. The number of relations no longer grows, and the lowest frequency until it starts to increase at this time, which shows that the adjacent relation of the model has become local completeness before this.

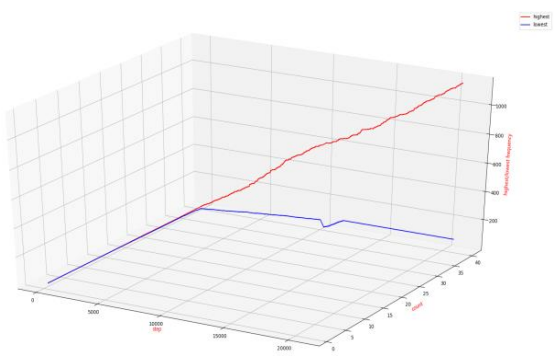

Fig. 3. A diagram of lowest frequency drift detection results.

B. Evaluation of feasibility

Because drift detection needs to know about the position of the drift point in advance, it is difficult to verify the feasibility of the detection results directly with the real data sets. Therefore, this paper references [17] to draw a Petri net model of the software development process adopting the PIPE tool, which is shown in Figure 4.

The model diagram contains four basic structures: sequence, selection, concurrency, and iteration. It also includes complex structures such as iteration mixed in concurrency and length-two loops mixed in concurrency. In the process of generating log by PIPE tool, we change the model in 12 change basic patterns according to [6] (CB, CD, CF, CM, CP, FR, LP, PL, PM, RE, RP, SW, respectively), and the position of the drift point was recorded during we change the model. At the same time, logs of five different lengths (800, 2400, 4000, 6400, 8000) are set, and there are five drift points in each log. Experimental results show that the algorithm can accurately detect the position of the first drift point of each log.

Because of space cause, the drift detection results of one of the change patterns (RP: replacing a segment) at the length of 2400 is given in Figure 5.

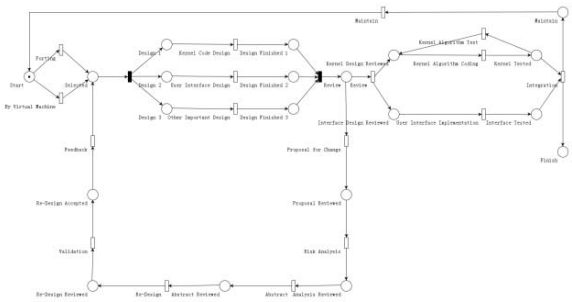

Fig. 4. A Petri net model diagram of the software development process.

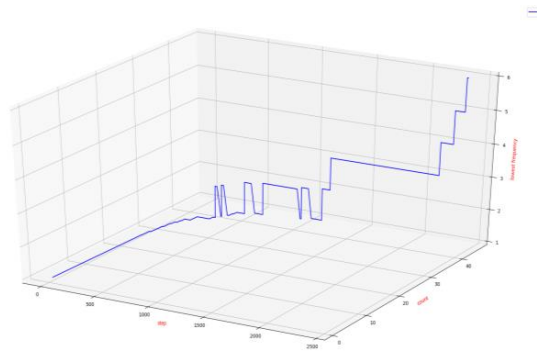

Fig. 5. The results of the RP change pattern.

In Figure 5, the positions of five drift points are detected by the algorithm, which is 332, 362, 533, 941, and 1017, respectively. The actual positions of the drift points are 332, 533, 941, 1230, and 1530. Three are accurately detected, and two are not.

VI. Conclusion

This paper proposes an algorithm for detecting the drift of the single-instance log in the software development process, striving to solve the problem that the drift in this process log cannot be directly detected by the existing drift detection algorithm due to the lack of case and activity attributes. The main contributions of this paper are summarized.

First, the LDA topic model is applied to extract the document of submission instruction of the event log to map activities, solve the problem of lacking activity attributes in the software development process log, and improve the traditional algorithm that human defines the mapping rules, and the number of mapped activities is small and fixed.

Secondly, the change of the lowest frequency of adjacent relation is used to detect the drift of a single-instance activity sequence, initially solve the problem the process log lacking case attributes and only one case information, which cannot be

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:40:55 UTC from IEEE Xplore. Restrictions apply.

directly detected by the existing drift detection algorithm.

The next step is to continue to study in-depth on how to accurately and comprehensively detect the drift point of the single-instance event log, and the problem of full matching of the activity mapping process.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (No. 61662085 , No. 61862065), the Yunnan Provincial Natural Science Foundation Fundamental Research Project (2019FB-16), the Yunnan University Data-Driven Software Engineering Provincial Science and Technology Innovation Team Project (2017HC012), the Yunnan University "Dong Lu Young-backbone Teacher" Training Pro-gram and the Yunnan University Education Department Science Research Fund Graduate Program (2019Y0008 , 2019Y0010).

REFERENCES

[1] Van Der Aalst W. Process mining: discovery, conformance and enhancement of business processes[M]. Heidelberg: Springer, 2011.

[2] Bose R P J C, van der Aalst W M P, Žliobaitė I, and Pechenizkiy M, Handling concept drift in process mining[C]//International Conference on Advanced Information Systems Engineering. Springer, Berlin, Heidelberg, 2011: 391-405.

[3] Zheng C, Wu X, Wen L, and Wang J, Detecting concept drift of process models from event logs[J]. Computer Integrated Manufacturing Systems, 2019, 25(04):44-50 (in Chinese).

[4] Bose R P J C, Van Der Aalst W M P, Žliobaitė I, and Pechenizkiy M, Dealing with concept drifts in process mining[J]. IEEE transactions on neural networks and learning systems, 2013, 25(1): 154-171.

[5] Martjushev J, Bose R P J C, van der Aalst W M P. Change point detection and dealing with gradual and multi-order dynamics in process mining[C]//International Conference on Business Informatics Research. Springer, Cham, 2015: 161-178.

[6] Maaradji A, Dumas M, La Rosa M, and Ostovar A, Fast and accurate business process drift detection[C]//International Conference on Business Process Management. Springer, Cham, 2016: 406-422.

[7] Kumar M, Thomas L, Basava A. Capturing the Sudden Concept Drift in Process Mining[C]//ATAED@ Petri Nets/ACSD. 2015: 132-143.

[8] Zheng C, Wen L, Wang J . Detecting Process Concept Drifts from Event Logs[C]// OTM Confederated International Conferences "On the Move to Meaningful Internet Systems". Springer, Cham, 2017.

[9] Lin L, Wen L, Zhou H, Pei J, and Zheng C, An approach for Process Concept Drift Detection based on Log Completeness[J]. Computer Integrated Manufacturing Systems, 2019, 25(04):87-95 (in Chinese).

[10] Zhu R, Li T, Mo Q, He Z, Yu Q, Wang Y. Data-Driven bilayer software process mining[J]. Ruan Jian Xue Bao/Journal of Software, 2018, 29(11): 3455−3483 (in Chinese).

[11] Blei D M, Ng A Y, Jordan M I. Latent dirichlet allocation[J]. Journal of machine Learning research, 2003, 3(Jan): 993-1022.

[12] Widmer G, Kubat M. Learning in the presence of concept drift and hidden contexts[J]. Machine learning, 1996, 23(1): 69-101.

[13] Lohmann N, Verbeek E, Dijkman R. Petri net transformations for business processes–a survey[M]//Transactions on petri nets and other models of concurrency II. Springer, Berlin, Heidelberg, 2009: 46-63.

[14] Zhu R, Li T, Mo Q, Dai F, Gao T, He Y, Sun X. Heuristic parallelized mining single firing sequence[J]. Computer Integrated Manufacturing Systems, 2016,22(2):330−342 (in Chinese).

[15] Li P, Lu F, Bao Y, Zeng Q, and Zhu G, Research on Drug Recommendation Approach Based on Medical Process Mining and Patient Signs[J]. Computer Integrated Manufacturing Systems, in press(in Chinese).

[16] Rubin V, Günther C W, Van Der Aalst W M P, Dongen EKBFV, Schafer W. Process mining framework for software processes[C]//International conference on software process. Springer, Berlin, Heidelberg, 2007: 169-181.

[17] Li T. An approach to modelling software evolution processes[M]. Tsinghua University Press, 2008.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:40:55 UTC from IEEE Xplore. Restrictions apply.